SUMMARY: NumPy-I

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what NumPy is and its importance
- Understand what array is
- Understand the creation of Numpy Arrays
- Understand the attributes of NumPy Array
- Understand the concept of indexing and slicing on NumPy arrays

KEY POINTS AND EXAMPLES:

Understand what NumPy is and its importance

NumPy, short for Numerical Python, is a fundamental library for scientific computing in Python.

NumPy is all about arrays. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.

NumPy is essential for performing efficient numerical computations and is the foundation of many other scientific libraries in Python.

NumPy provides various statistical functions such as mean, median, standard deviation, and variance, which are essential for performing basic statistical analysis on data. This helps in understanding the data distribution and identifying patterns.

Although NumPy itself is not a visualization library, it works seamlessly with libraries like Matplotlib. Data stored in NumPy arrays can be easily visualized using Matplotlib, enabling analysts to create plots, histograms, and other graphical representations of data.

Installing NumPy

NumPy can be installed by typing following command:

```
conda install numpy
```

or

```
pip install numpy
```

Importing NumPy

To access NumPy and its functions import it in Python code like this:

```
import numpy as np
```

We shorten the imported name to np for better readability of code using NumPy. This is a widely adopted convention that makes your code more readable for everyone working on it.

Understand what array is

We have discussed about various data types like list, tuple, and dictionary. In this session we will discuss another datatype 'Array'.

An array is a data type used to store multiple values using a single identifier (variable name).

An array contains an ordered collection of data elements where each element is of the same type and can be referenced by its index (position).

The important characteristics of an array are:

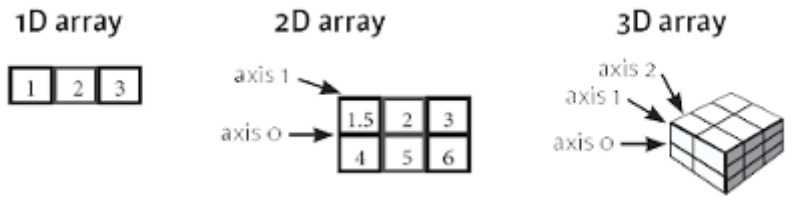
- Each element of the array is of same data type, though the values stored in them may be different.
- The entire array is stored contiguously in memory. This makes operations on array fast.
- Each element of the array is identified or referred using the name of the Array along with the index of that element, which is unique for each element. The index of an element is an integral value associated with the element, based on the element's position in the array.

For example consider an array with 5 numbers: [10, 9, 99, 71, 90]
Here, the 1st value in the array is 10 and has the index value [0] associated with it; the 2nd value in the array is 9 and has the index value [1] associated with it, and so on. The last value (in this case the 5th value) in this array has an index [4].

NumPy Array

NumPy arrays are used to store lists of numerical data, vectors and matrices.Numpy arrays are mutable. The NumPy library has a large set of routines (built-in functions) for creating, manipulating, and transforming NumPy arrays. Python language also has an array data

structure, but it is not as versatile, efficient and useful as the NumPy array. The NumPy array is officially called ndarray but commonly known as array.



Note: In the rest of the chapter, we will be referring to NumPy array whenever we use "array".

Understand the creation of Numpy Arrays

There are several ways to create arrays.
To create an array and to use its methods, first we need to import the NumPy library.

Creation of NumPy Arrays from List

We can create an array directly from a list or a tuple using np.array().

```
import numpy as np
# From a list
array_from_list = np.array([1, 2, 3, 4])
print(array_from_list)

# From a tuple
array_from_tuple = np.array((1, 2, 3, 4))
print(array_from_tuple)
```

The output will be:

```
[1 2 3 4]
[1 2 3 4]
```

- array_from_list: This line creates a NumPy array from a Python list [1, 2, 3, 4]. The resulting array has the same elements as the list.
- array_from_tuple: This line creates a NumPy array from a Python tuple (1, 2, 3, 4). The resulting array has the same elements as the tuple.

Note: A common mistake occurs while passing argument to array() if we forget to put square brackets. Make sure only a single argument containing list of values is passed.

```
#incorrect way
a = np.array(1,2,3,4)
print(a)
```

The output will be:

```
TypeError: array() takes from 1 to 2 positional arguments but 4
were given
```

We can use Python's built-in type() function to determine the type of a NumPy array object itself.

```
import numpy as np

# Create an array
array = np.array([1, 2, 3])

# Check the type of the array object
print("Type of array:", type(array))
```

The output will be:

```
Type of array: <class 'numpy.ndarray'>
```

Creating 1-D Array: An array with only single row of elements is called 1-D array. 1-D array can be created from lists, tuples, or using various NumPy functions.

From a List

```
import numpy as np

# From a list
array_from_list = np.array([1, 2, 3, 4])
print("1-D array from list:", array_from_list)
```

The output will be:

```
1-D array from list: [1 2 3 4]
```

From a Tuple

```
import numpy as np

# From a tuple
array_from_tuple = np.array((1, 2, 3, 4))
print("1-D array from tuple:", array_from_tuple)
```

The output will be:

```
1-D array from tuple: [1 2 3 4]
```

Creating 2-D Array: A 2-D array, also known as a two-dimensional array or matrix, that represents a collection of elements organized into rows and columns.

```
import numpy as np

# Creating a 2-D array from a list of lists
array_2d = np.array([[1, 2, 3],
                     [4, 5, 6],
                     [7, 8, 9]])

print("2-D array (matrix):\n", array_2d)
```

The output will be:

```
2-D array (matrix):
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In this example:

- array_2d is a 3x3 matrix (3 rows and 3 columns).

For a 2D NumPy array, the axes are explained as follows:
Axis 0: Refers to the first dimension of an array. For a 2D array, it represents rows.
Axis 1: Refers to the second dimension of an array. For a 2D array, it represents columns.

Creating 3-D Array: A 3-D array, also known as a three-dimensional array, extends the concept of a 2-D array by adding another dimension. It can be visualized as a collection of 2-D arrays stacked along a third axis.

```
import numpy as np

# Creating a 3-D array from a list of lists of lists
array_3d = np.array([[[1, 2],
                      [3, 4]],

                     [[5, 6],
                      [7, 8]]])

print("3-D array:\n", array_3d)
```

The output will be:

```
3-D array:
 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

In this example:

- array_3d is a 2x2x2 array (2 slices, each containing a 2x2 matrix).

For a 3D NumPy array, the axes are explained as follows:
Axis 0: Refers to the first dimension of the 3D array, which represents the matrices.
Axis 1: Refers to the second dimension of the 3D array, which represents the rows within each matrix.
Axis 2: Refers to the third dimension of the 3D array, which represents the columns within each matrix..

Specifying Data Types: We can specify data type (integer, float, etc.) while creating array using dtype as an argument to array(). This will convert the data automatically to the mentioned type.

In the following example, nested list of integers are passed to the array function. Since data type has been declared as float, the integers are converted to floating point numbers.

```
import numpy as np

# Nested list of integers
nested_list = [[1, 2, 3],
```

```
                [4, 5, 6],
                [7, 8, 9]]
# Creating a NumPy array with float dtype
array_float = np.array(nested_list, dtype=float)

print("Array with float dtype:\n", array_float)
```

The output will be:

```
Array with float dtype:
 [[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
```

When you create a NumPy array with elements of different data types, NumPy tries to find a common data type that can accommodate all the elements.

```
import numpy as np

array2 = np.array([5, -7.4, 'a', 7.2])
print(array2)
```

The output will be:

```
['5' '-7.4' 'a' '7.2']
```

NumPy arrays are designed to have elements of the same data type for efficient computation and memory usage. When you provide elements of different types (int, float, str, float), NumPy tries to find a common data type that all elements can be converted to.

NumPy determines that the array can be best represented as an array of strings (dtype='<U32', Unicode strings of up to 32 characters).

If you want to maintain the original data types or ensure elements are converted to a specific type, you can explicitly specify the dtype parameter when creating the array:

```
import numpy as np

# Specify dtype explicitly
array2 = np.array([5, -7.4, 'a', 7.2], dtype=object)
```

```
print(array2)
```

The output will be:

```
[5 -7.4 'a' 7.2]
```

If Numpy cannot find a common data type that can accurately represent all elements, NumPy will upcast the elements to a more general data type that can represent all of them.

```
import numpy as np

array3 = np.array([[2.4, 3],
                   [4.91, 7],
                   [0, -1]])

print(array3)
```

The output will be:

```
[[ 2.4   3.  ]
 [ 4.91  7.  ]
 [ 0.   -1.  ]]
```

Understand the attributes of NumPy Array

(Comment for instructor: Instructor, just give basic information on the attributes of a NumPy array without delving too deeply.)

Some important attributes of a NumPy ndarray object are:

1. **ndarray.ndim:** The ndarray.ndim attribute in NumPy provides information about the number of dimensions (axes) of the array. Understanding ndarray.ndim is crucial for determining the structure and shape of NumPy arrays, which is foundational for performing operations and manipulating data effectively.

```
import numpy as np

# 1-dimensional array (vector)
arr1d = np.array([1, 2, 3])
```

```
print("Array:", arr1d)
print("Number of dimensions:", arr1d.ndim)

# 2-dimensional array (matrix)
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:")
print(arr2d)
print("Number of dimensions:", arr2d.ndim)

# 3-dimensional array
arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print("Array:")
print(arr3d)
print("Number of dimensions:", arr3d.ndim)
```

The output will be:

```
Array: [1 2 3]
Number of dimensions: 1
Array:
[[1 2 3]
 [4 5 6]]
Number of dimensions: 2
Array:
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
Number of dimensions: 3
```

In this example:

- 1-dimensional array (arr1d):
  - Contains a single sequence of elements (e.g., [1, 2, 3]).
  - ndarray.ndim returns 1, indicating it has one dimension.
- 2-dimensional array (arr2d):
  - Represents a matrix with rows and columns (e.g., [[1, 2, 3], [4, 5, 6]]).
  - ndarray.ndim returns 2, indicating it has two dimensions.
- 3-dimensional array (arr3d):
  - Contains arrays nested within arrays, forming a 3D structure (e.g., [[[1, 2], [3, 4]], [[5, 6], [7, 8]]]).
  - ndarray.ndim returns 3, indicating it has three dimensions.

2. **ndarray.shape**: The ndarray.shape attribute in NumPy provides a tuple that represents the size of the array along each dimension.

```
import numpy as np

# Create a 1D NumPy array
arr_1d = np.array([1, 2, 3, 4, 5])

# Access the shape attribute for 1D array
shape_tuple_1d = arr_1d.shape

# Print the shape tuple for 1D array
print("Shape of the 1D array:", shape_tuple_1d)

# Create a 2D NumPy array
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Access the shape attribute for 2D array
shape_tuple_2d = arr_2d.shape

# Print the shape tuple for 2D array
print("Shape of the 2D array:", shape_tuple_2d)
```

The output will be:

```
Shape of the 1D array: (5,)
Shape of the 2D array: (2, 3)
```

In this example:

- arr_1d is a one-dimensional array with 5 elements. The shape attribute returns a tuple (5,), indicating that the array has one dimension with a size of 5.
- arr is a 2-dimensional NumPy array ([[1, 2, 3], [4, 5, 6]]).
- arr.shape returns (2, 3), indicating that arr has 2 rows and 3 columns.

```
import numpy as np

# Create a 3-dimensional NumPy array
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10],
[11, 12]]])
```

```
# Access the shape attribute
shape_tuple = arr_3d.shape

# Print the shape tuple
print("Shape of the 3D array:")
print(shape_tuple)
```

The output will be:

```
Shape of the 3D array:
(3, 2, 2)
```

In this example:

- arr_3d is a 3-dimensional NumPy array created using np.array().
- The array arr_3d has three dimensions:
  - First dimension (axis 0): Size 3 indicates there are 3 matrices.
  - Second dimension (axis 1): Size 2 indicates each matrix has 2 rows.
  - Third dimension (axis 2): Size 2 indicates each matrix has 2 columns.
- Three Matrices: This means arr_3d is composed of 3 separate 2-dimensional arrays (matrices). Each matrix is structured with 2 rows and 2 columns.

3. **ndarray.size:** The ndarray.size attribute in NumPy provides the total number of elements in the array. This attribute is calculated as the product of the elements of the ndarray.shape tuple, which represents the size of the array along each dimension.

```
import numpy as np

# Create a 1D NumPy array
arr_1d = np.array([1, 2, 3, 4, 5])

# Access the size attribute for the 1D array
size_1d = arr_1d.size

# Print the size of the 1D array
print("Size of the 1D array:", size_1d)

# Create a 2D NumPy array
arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

# Access the size attribute for the 2D array
```

```
size_2d = arr_2d.size

# Print the size of the 2D array
print("Size of the 2D array:", size_2d)
```

The output will be:

```
Size of the 1D array: 5
Size of the 2D array: 6
```

In this example,

- arr_1d is a one-dimensional array with 5 elements. The size attribute returns 5, indicating that the array contains 5 elements in total.
- arr is a 2-dimensional NumPy array ([[1, 2, 3], [4, 5, 6]]).
- arr.size returns 6, which is the total number of elements in the array.
- Calculation: The shape of arr is (2, 3), so the size is 2 * 3 = 6.

```
import numpy as np

# Create a 3-dimensional NumPy array
arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]], [[9, 10],
[11, 12]]])

# Access the size attribute
array_size = arr_3d.size

# Print the size of the array
print("Size of the 3D array:", array_size)
```

The output will be:

```
Size of the 3D array: 12
```

In this example:

- arr_3d is a 3-dimensional NumPy array created using np.array()
- arr_3d.size returns 12, which is the total number of elements in the array.
- Calculation: The array arr_3d contains 3 matrices, each with 2 rows and 2 columns, resulting in 3 * 2 * 2 = 12 elements.

4. **ndarray.dtype:** The ndarray.dtype attribute in NumPy represents the data type of the elements contained within the array. In NumPy arrays, all elements must have the

same data type.

```
import numpy as np

# Create a NumPy array with integers
arr_int = np.array([1, 2, 3, 4, 5])
print("Data type of elements in arr_int:", arr_int.dtype)

# Create a NumPy array with floating point numbers
arr_float = np.array([1.0, 2.5, 3.7, 4.2, 5.9])
print("Data type of elements in arr_float:", arr_float.dtype)

# Create a NumPy array with Unicode strings
arr_str = np.array(['apple', 'banana', 'cherry'])
print("Data type of elements in arr_str:", arr_str.dtype)

# Create a NumPy array with mixed data types
array_ = np.array([5, -7.4, 'a', 7.2])
print("Data type of elements in array_:", array_.dtype)
```

The output will be:

```
Data type of elements in arr_int: int32
Data type of elements in arr_float: float64
Data type of elements in arr_str: <U6
Data type of elements in array_: <U32
```

- In this example:
  arr_int.dtype: Returns int32, indicating that all elements in arr_int are 32-bit integers.
- arr_float.dtype: Returns float64, indicating that all elements in arr_float are 64-bit floating point numbers.
- arr_str.dtype: Returns <U6, indicating that all elements in arr_str are Unicode strings of up to 6 characters.
- array_.dtype: Returns <U32, indicating that all elements in array_ are Unicode strings of up to 32 characters.

5. **ndarray.itemsize:** The ndarray.itemsize attribute in NumPy represents the size in bytes of each element in the array. It provides information about how much memory each element occupies based on its data type.

```python
import numpy as np
# Create NumPy arrays with different data types
arr_int = np.array([1, 2, 3, 4, 5], dtype=int)
arr_float = np.array([1.0, 2.5, 3.7, 4.2, 5.9], dtype=float)
arr_str = np.array(['apple', 'banana', 'cherry'], dtype=str)

# Print item sizes of each array
print("Item size of arr_int:", arr_int.itemsize, "bytes")
print("Item size of arr_float:", arr_float.itemsize, "bytes")
print("Item size of arr_str:", arr_str.itemsize, "bytes")
```

The output will be:

```
Item size of arr_int: 4 bytes
Item size of arr_float: 8 bytes
Item size of arr_str: 24 bytes
```

In this example:

- arr_int.itemsize: Returns 4 bytes, because each element in arr_int (dtype int) occupies 4 bytes of memory.
- arr_float.itemsize: Returns 8 bytes, indicating that each element in arr_float (dtype float) occupies 8 bytes.
- arr_str.itemsize: Returns 32 bytes, as each element in arr_str (dtype str) is a string with a maximum length of 32 characters (default for Unicode strings).

6. **ndarray.data**: The ndarray.data attribute in NumPy provides a direct buffer to the memory location where the array's data is stored.

```python
import numpy as np

# Create a NumPy array
arr = np.array([1, 2, 3, 4, 5])

# Access the raw data buffer using ndarray.data
data_buffer = arr.data

# Print the data buffer and its type
print("Data buffer:", data_buffer)
print("Type of data buffer:", type(data_buffer))
```

The output will be:

```
Data buffer: <memory at 0x0000022CAB6A6B80>
Type of data buffer: <class 'memoryview'>
```

Understand the concept of indexing and slicing on NumPy arrays

Indexing in NumPy Arrays

Indexing in NumPy arrays is similar to indexing in Python lists. It allows you to access individual elements or subsets of elements in an array. NumPy arrays can be indexed using integer indices or boolean masks.

1. Integer Indexing:

- You can access individual elements using their positional index, just like in Python lists.
- For 1D arrays, you specify a single index.
- For 2D arrays, you specify two indices separated by a comma (row, column).
- For higher-dimensional arrays, you specify indices for each dimension.

```
import numpy as np

# 1D array
arr_1d = np.array([1, 2, 3, 4, 5])
print(arr_1d[2])

# 2D array
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr_2d[1, 2])

# 3D array
arr_3d = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11,
12]], [[13, 14, 15], [16, 17, 18]]])
print(arr_3d[1, 0, 2])
```

The output will be:

```
3
6
9
```

- 1D array: arr_1d[2] accesses the element at index 2 (3rd element) in the 1D array arr_1d, resulting in the output 3.

- 2D array: arr_2d[1, 2] accesses the element at row index 1 and column index 2 in the 2D array arr_2d, resulting in the output 6.
- 3D array: arr_3d[1, 0, 2] accesses the element at the first matrix (1) in the 3D array arr_3d, then the row index 0, and finally the column index 2, resulting in the output 9.

In NumPy arrays, indexing works similarly across dimensions:

- For 2D arrays, use [row_index, column_index].
- For 3D arrays, extend this to [matrix_index, row_index, column_index].

2. Boolean Indexing (Masking):
   - We can use boolean arrays (arrays of True/False values) to select elements from an array.
   - The boolean array must have the same shape as the array being indexed.
   - Elements are selected where the boolean array is True.

```
import numpy as np

# 1D array
arr_1d = np.array([10, 20, 30, 40, 50])
mask_1d = np.array([True, False, True, False, True])
print("1D array with mask:", arr_1d[mask_1d])
# 2D array
arr_2d = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
mask_2d = np.array([[True, False, True], [False, True, False],
[True, False, True]])
print("2D array with mask:\n", arr_2d[mask_2d])
# 3D array
arr_3d = np.array([[[10, 20], [30, 40]], [[50, 60], [70, 80]],
[[90, 100], [110, 120]]])
mask_3d = np.array([[[True, False], [False, True]], [[True,
False], [False, True]], [[True, False], [False, True]]])
print("3D array with mask:\n", arr_3d[mask_3d])
```

The output will be:

```
1D array with mask: [10 30 50]
2D array with mask:
 [10 30 50 70 90]
3D array with mask:
 [ 10  40  50  80  90 120]
```

- 1D array: The boolean mask mask_1d is applied to the 1D array arr_1d, resulting in an array that only includes the elements where the mask is True.
- 2D array: The boolean mask mask_2d is applied to the 2D array arr_2d. The output is a flattened array containing elements of arr_2d where the corresponding element in mask_2d is True.
- 3D array: The boolean mask mask_3d is applied to the 3D array arr_3d. Similarly, the output is a flattened array containing elements of arr_3d where the corresponding element in mask_3d is True.

Note: A flattened array is a one-dimensional array that contains all the elements of a multi-dimensional array. When you flatten an array, you take all the elements from the original multi-dimensional structure and list them out in a single continuous sequence, losing the original shape in the process.

Slicing in NumPy Arrays

Slicing in NumPy arrays is similar to slicing in Python lists, but it provides more powerful and flexible ways to select subsets of elements from an array.

1. Basic Slicing:
    - We can use the colon : to select a range of elements in an array.
    - The syntax is start:stop:step, where start is the starting index (inclusive), stop is the ending index (exclusive), and step is the step size.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(arr[2:6])
print(arr[::2])
```

The output will be:

```
[3 4 5 6]
[1 3 5 7 9]
```

In this example:
- arr[2:6]: This slices the array from index 2 to index 5 (6 is excluded).
- arr[::2]: This slices the array from the beginning to the end with a step of 2.

2. Multi-dimensional Slicing:

- For multi-dimensional arrays, we can specify slices for each dimension separated by commas.
- we can also use the ... ellipsis to represent all remaining dimensions.

```
import numpy as np

# Create a 2D NumPy array
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12],
                   [13, 14, 15, 16]])

# Slice to get a subarray
subarray = arr_2d[1:3, 1:3]
print("Subarray from 2D array:")
print(subarray)
```

The output will be:

```
Subarray from 2D array:
[[ 6  7]
 [10 11]]
```

In this example:

- 1:3 for the rows: This means we are selecting rows starting from index 1 up to (but not including) index 3.
- 1:3 for the columns: This means we are selecting columns starting from index 1 up to (but not including) index 3.
- Rows Selection
  - Index 1 row: [5, 6, 7, 8]
  - Index 2 row: [9, 10, 11, 12]
- For row [5, 6, 7, 8]: Selecting columns at index 1 and 2 gives [6, 7].
- For row [9, 10, 11, 12]: Selecting columns at index 1 and 2 gives [10, 11].

```
import numpy as np

# Create a 2D NumPy array
array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
# Slice the first two rows
print("Slice first two rows:\n", array_2d[:2])

# Slice the last two columns
print("Slice last two columns:\n", array_2d[:, 1:])

# Slice the bottom-right 2x2 block
print("Slice bottom-right 2x2 block:\n", array_2d[1:, 1:])
```

The output will be:

```
Slice first two rows:
 [[1 2 3]
 [4 5 6]]
Slice last two columns:
 [[2 3]
 [5 6]
 [8 9]]
Slice bottom-right 2x2 block:
 [[5 6]
 [8 9]]
```

```
import numpy as np

# Create a 3D NumPy array
arr_3d = np.array([[[ 1,  2,  3], [ 4,  5,  6]],
                   [[ 7,  8,  9], [10, 11, 12]],
                   [[13, 14, 15], [16, 17, 18]]])

# Slice to get a subarray
subarray_3d = arr_3d[0:2, 1, 0:2]
print("Subarray from 3D array:")
print(subarray_3d)
```

The output will be:

```
Subarray from 3D array:
[[ 4  5]
 [10 11]]
```

In this example:

- 0:2: This selects the first two matrices from the 3D array (the 0th and 1st matrices).
- 1: This selects the second row (index 1) from the selected matrices.
- 0:2: This selects the first two columns (index 0 and 1) from the selected rows.

(Comment for instructor: Instructor, give the basic information on Ellipsis, how it generates the output, and explain that ellipsis in NumPy arrays can be important in certain contexts, especially when dealing with high-dimensional arrays.)

To select all elements from a specific dimension, we can use .... For example, if you want to select all matrices (axis=0) and all their elements:

```
import numpy as np

arr_3d = np.array([[[1, 2, 3], [4, 5, 6]],
                   [[7, 8, 9], [10, 11, 12]],
                   [[13, 14, 15], [16, 17, 18]]])


subarray = arr_3d[..., :]
print("Subarray with ellipsis:")
print(subarray)
```

The output will be:

```
Subarray with ellipsis:
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]

 [[13 14 15]
  [16 17 18]]]
```

Here, ... represents all remaining dimensions after specifying the first dimension (which is matrices in this case). So, [..., :] means:

- ...: All matrices in the first dimension (axis=0).
- :: All rows and columns in the second and third dimensions (axis=1 and axis=2).

```
import numpy as np

arr_3d = np.array([[[1, 2, 3], [4, 5, 6]],
                   [[7, 8, 9], [10, 11, 12]],
                   [[13, 14, 15], [16, 17, 18]]])
subarray = arr_3d[:, 0, ...]
print(subarray)
```

The output will be:

```
[[ 1  2  3]
 [ 7  8  9]
 [13 14 15]]
```

In this example:

- This arr_3d array has the following structure:
  - Dimension 1 (axis=0): Contains 3 matrices.
  - Dimension 2 (axis=1): Each matrix has 2 rows.
  - Dimension 3 (axis=2): Each row has 3 elements.
- Here, [:, 0, ...] means:
  - :: All matrices in the first dimension (axis=0).
  - 0: The first row in the second dimension (axis=1).
  - ...: All remaining dimensions (in this case, all columns in the third dimension).

Understand operations on arrays

NumPy provides a wide range of operations that can be performed on arrays.

Arithmetic Operations

Arithmetic operations on NumPy arrays are fast and simple. When we perform a basic arithmetic operation like addition, subtraction, multiplication, division etc. on two arrays, the operation is done on each corresponding pair of elements.
For instance, adding two arrays will result in the first element in the first array to be added to the first element in the second array.

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
# Element-wise addition
c = a + b
print("Addition:")
print(c)

# Element-wise subtraction
d = b - a
print("Subtraction:")
print(d)

# Element-wise multiplication
e = a * b
print("Multiplication:")
print(e)

# Element-wise division
f = b / a
print("Division:")
print(f)

# Element-wise modulus
g = b % a
print("Modulus:")
print(g)

# Element-wise exponentiation
h = a ** 2
print("Exponentiation:")
print(h)

# Element-wise floor division
i = b // a
print("Floor Division:")
print(i)
```

The output will be:

```
Addition:
[5 7 9]
Subtraction:
[3 3 3]
```

```
Multiplication:
[ 4 10 18]
Division:
[4.  2.5 2. ]
Modulus:
[0 1 0]
Exponentiation:
[1 4 9]
Floor Division:
[4 2 2]
```

Note: It is important to note that for element-wise operations, size of both arrays must be same. That is, array1.shape must be equal to array2.shape.